STOCKY AI

Proactive AI Business Partner for Malaysian Produce Wholesalers

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026 | Version 1.0

umhackathon@um.edu.my

Powered by Z.ai | ilmu-glm-5.1 | YTL AI Labs

Team Name	Grabbeez
Repository	https://github.com/Vaarshh17/StockyAI
Domain	AI for Economic Empowerment & Decision Intelligence
Document Version	1.0
Date	24 April 2026

Table of Contents

Table of Contents 2
Introduction 3
- Purpose 3
- Background 3
- Previous Version / Changes 3

1. Target Stakeholders 4
2. System Architecture & Design 5
- 2.1 High-Level Architecture Overview 5
- 2.2 Component Architecture 5
- 2.3 LLM as Service Layer — Z.ai Integration 7
  - Prompt Construction 7
  - What Goes Into the Context Window 7
  - Response Parsing 7
  - Token Limit Enforcement 8
- 2.4 Dependency Diagram — API Calls Between Components 8
- 2.6 Technology Stack 11

3. Key Data Flows 12
- 3.1 Data Flow Diagram — System Level 12
- 3.2 Normalised Database Schema (3NF) 12

4. Functional Requirements & Scope 14
- 4.1 Minimum Viable Product (MVP) 14
- 4.2 Non-Functional Requirements 14
- 4.3 Out of Scope / Future Enhancements 15

5. Monitoring, Evaluation, Assumptions & Dependencies 16
- 5.1 Grayscale Rollout & A/B Testing Strategy 16
- 5.2 Emergency Rollback & Golden Release 16
- 5.3 Priority Matrix 16
- 5.4 Monitoring Plan 17
- 5.5 Assumptions 17
- 5.6 External Dependencies 18

6. Project Management & Team Contributions 19
- 6.1 Project Timeline (10-Day Sprint) 19
- 6.2 Team Member Roles 19

Introduction

This document presents the System Analysis Documentation (SAD) for Stocky AI — a proactive AI agent for Malaysian fruit and vegetable wholesalers, developed for UMHackathon 2026.

Purpose

The SAD defines the technical architecture, data flows, component interactions, and design decisions behind Stocky AI. It serves as the primary reference for developers, testers, and evaluators to understand how the system is built, how data moves through it, and how the Z.ai ilmu-glm-5.1 model is integrated as the core reasoning engine.

Key elements covered in this document:

- Architecture — event-driven Telegram bot with async Python backend and Supabase PostgreSQL
- Data Flows — how trader input moves through OCR, voice, and text channels into the agent loop and database
- Model Integration — how prompts are constructed, sent to ilmu-glm-5.1, and how tool-call responses are parsed
- Database Schema — 7-table normalised schema in 3NF
- Functional & Non-Functional Requirements — MVP scope and system quality targets
- Monitoring, Rollout & Risk — deployment strategy, priority matrix, and external dependencies

Background

Stocky AI was built from scratch during UMHackathon 2026 to solve a real operational problem: Malaysian wet market wholesalers make high-stakes, time-sensitive decisions (buy/sell/price) using only memory and paper records. The system was designed with one architectural principle above all others: every recommendation must be grounded in live data retrieved from tools — the model must never answer from training knowledge alone.

Previous Version / Changes

This is Version 1.0 — a greenfield build. There is no previous version. Major architectural decisions introduced in this version:

- Agentic tool-calling loop (vs. simple prompt-response chatbot)
- Proactive scheduler (6 autonomous jobs firing without user prompt)
- Approval → execution loop with WhatsApp deep link generation
- Local OCR and transcription (EasyOCR + faster-whisper) to eliminate vision API dependency

1. Target Stakeholders

Stakeholder	Role	Expectations
Wholesaler (Primary User)	Operates daily at Pasar Borong. Manages inventory, buys from suppliers, sells to retailers on credit.	Proactive morning brief at 3:30 AM. One-tap order execution. Zero data-entry for deliveries (photo). Alerts only when action is needed — silence otherwise.
Supplier (Indirect)	Receives order messages via WhatsApp. Does not interact with the system directly.	Receives pre-written order messages in their preferred language (Malay/Mandarin) via wa.me deep link.
Retail Buyer (Indirect)	Buys produce on credit from the wholesaler. Does not interact with the system.	Credit is tracked accurately. Payment reminders are sent in their language.
Development Team	Builds and maintains the platform. Each module owned by a specific team member.	Clean API contracts between layers. Modular architecture. Clear tool schemas. Async DB layer with typed return dicts.
QA Team	Validates agent loop behaviour, tool routing, scheduler jobs, and fallback states.	Comprehensive test suite with in-memory SQLite, mock LLM fixtures, and per-tool unit tests. conftest.py with shared fixtures.

2. System Architecture & Design

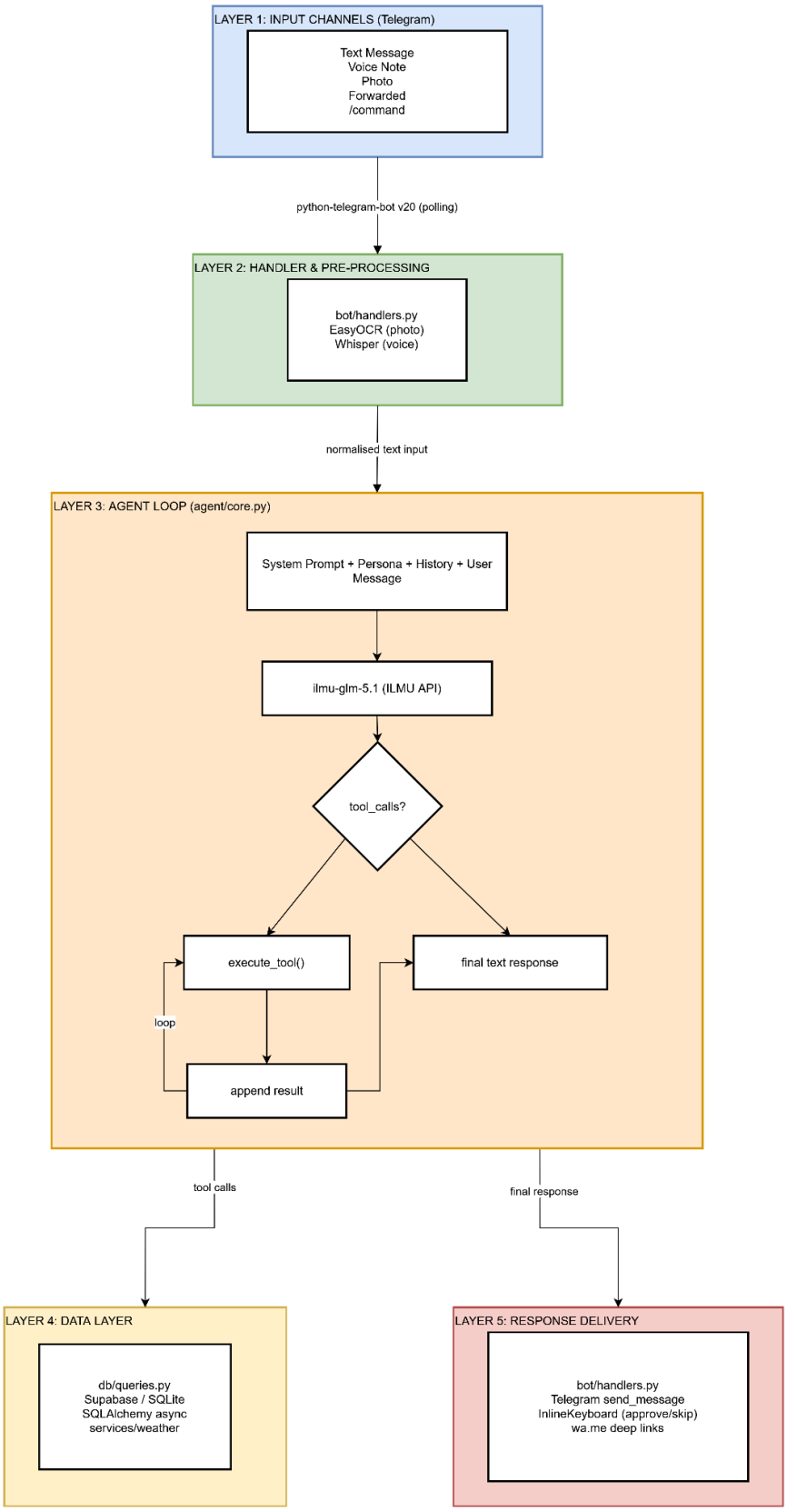
2.1 High-Level Architecture Overview

Type	Detail
System	Telegram Bot (cross-platform: Android, iOS, Desktop Web)
Architecture	Event-driven, single-service Python backend with async ORM and APScheduler
AI Engine	ilmu-glm-5.1 via ILMU API (Z.ai / YTL AI Labs) — OpenAI-compatible endpoint at api.ilmu.ai/v1
Database	Supabase PostgreSQL (production) / SQLite (local development) — accessed via SQLAlchemy async ORM
Local ML	EasyOCR (image text extraction) + faster-whisper (voice transcription) — both run in-process, no external API
Scheduler	APScheduler AsyncIOScheduler — 6 cron jobs, Asia/Kuala_Lumpur timezone
Deployment	Cloud VM or local Python process — polling mode, no public HTTPS endpoint required

Stocky AI is structured as a single Python service — not a microservices architecture. This is a deliberate choice for the hackathon: a monolith is faster to build, easier to test, and has no network latency between components. The service is stateless per-request (all state in DB + in-memory dicts), making it straightforward to scale horizontally in production.

2.2 Component Architecture

The system is organised into five distinct layers:


LAYER 1: INPUT CHANNELS (Telegram)
Text Message
Voice Note
Photo
Forwarded
/command
python-telegram-bot v20 (polling)
LAYER 2: HANDLER & PRE-PROCESSING
bot/handlers.py
EasyOCR (photo)
Whisper (voice)
normalised text input
LAYER 3: AGENT LOOP (agent/core.py)
System Prompt + Persona + History + User Message
ilmu-glm-5.1 (ILMU API)
tool_calls?
execute_tool()
final text response
loop
append result
tool calls
final response
LAYER 4: DATA LAYER
db/queries.py
Supabase / SQLite
SQLAlchemy async
services/weather
LAYER 5: RESPONSE DELIVERY
bot/handlers.py
Telegram send_message
InlineKeyboard (approve/skip)
wa.me deep links

2.3 LLM as Service Layer — Z.ai Integration

Prompt Construction

Every call to ilmu-glm-5.1 assembles a messages array with three parts:

- **System message:** build_profile_block(persona) [name, language, commodities, city] prepended to BASE_SYSTEM_PROMPT [mandatory tool chains, alert thresholds, response formats, communication rules]. Combined at call time via get_system_prompt(persona).
- **History:** Last N turns of real user conversation retrieved from in-memory _history dict (keyed by user_id). Proactive scheduler calls pass save_history=False — their prompts are never stored, preventing synthetic 'user' messages from polluting conversation context.
- **User message:** Normalised text input. For forwarded messages, an INSTRUCTION block is prepended forcing the model to call tools before responding. For OCR, the extracted text is wrapped in a parsing instruction.

What Goes Into the Context Window

Component	Approx. Tokens	Notes
Persona block (USER PROFILE)	~80	Name, language, commodities, city — injected every call
BASE_SYSTEM_PROMPT	~900	Tool chains, alert rules, response formats, communication rules
Conversation history	~600 (10 turns)	Capped at last 10 turns to prevent context overflow
User message	~50–400	Text, voice transcript, or OCR instruction
Tool results (per iteration)	~200–400 each	JSON serialised; up to 3 tools per iteration × 6 iterations max
Total (morning brief, 4 tools)	~2,400 input	max_tokens=3000 for output

Response Parsing

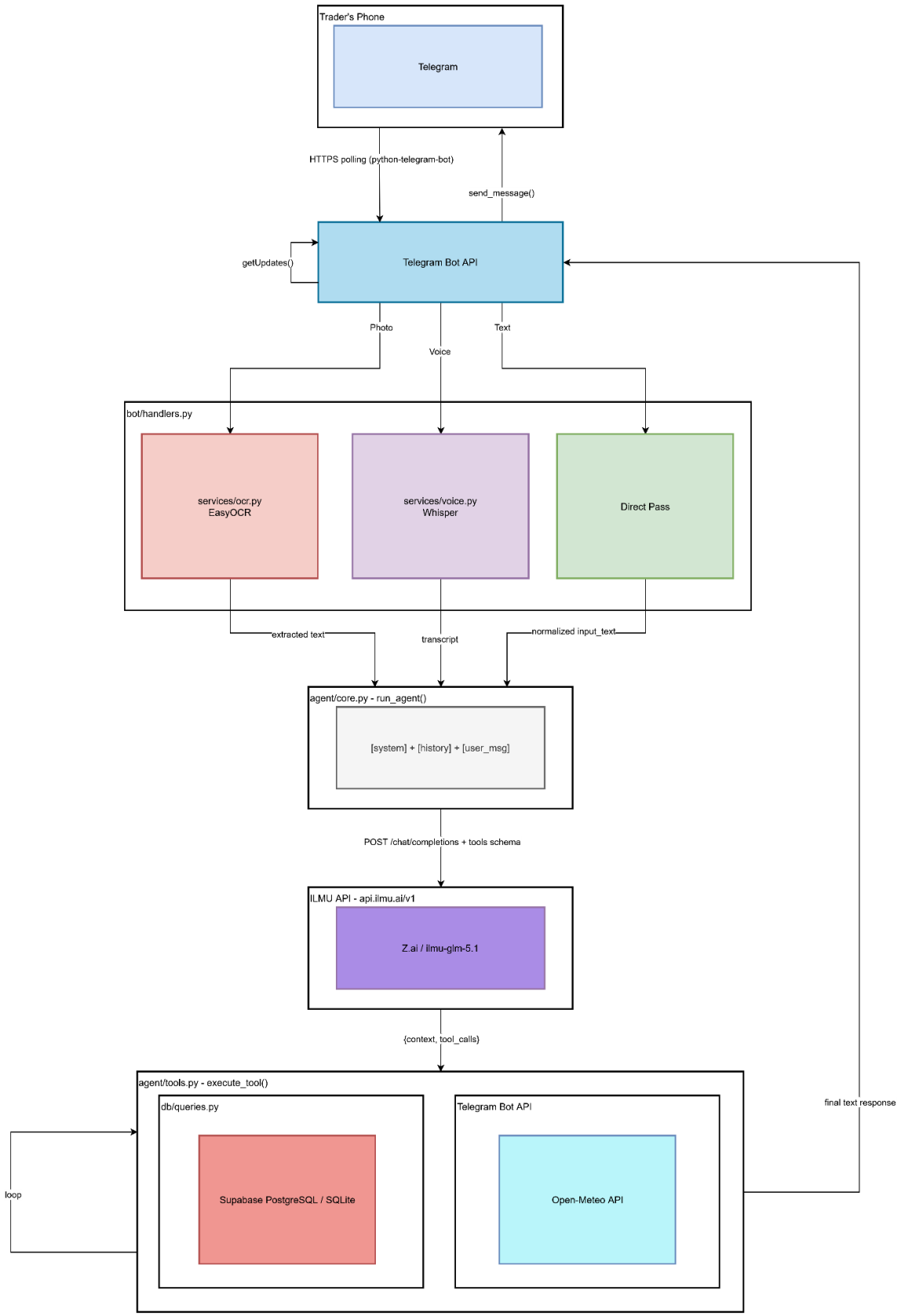
call_llm() in services/glm.py receives the raw OpenAI-format response and normalises it into a flat dict: {"content": str | None, "tool_calls": list | None}. The agent loop in core.py checks tool_calls first — if present, each is executed and its result appended as a role:tool message. If absent, the content is the final answer.

Tool call arguments are parsed with json.loads(tc["function"]["arguments"]), then filtered through per-tool allowlists in execute_tool() to prevent TypeErrors from hallucinated fields. Tool results are serialised with json.dumps(..., default=str) to handle date objects.

Token Limit Enforcement

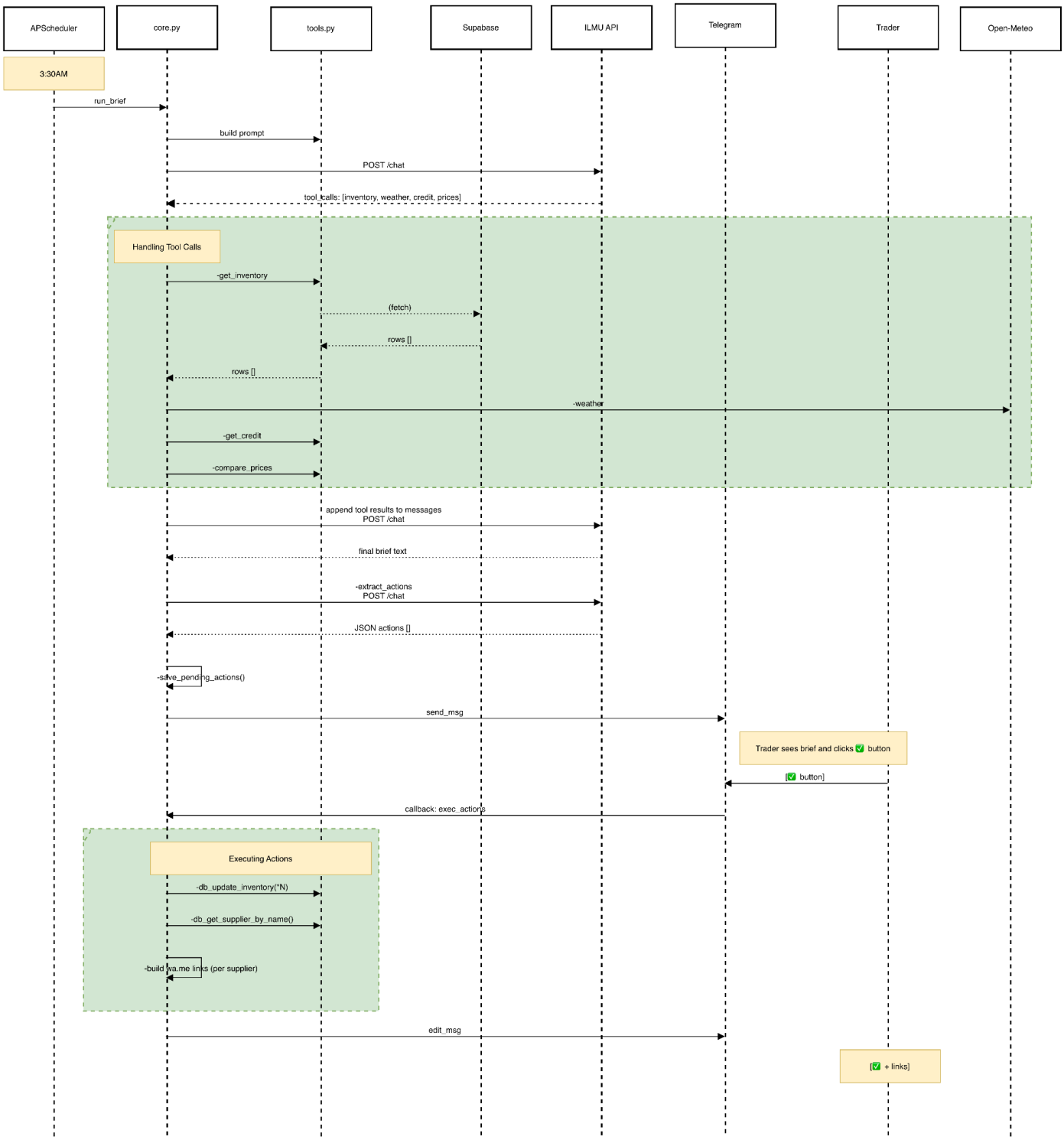
Token limits are enforced at two points: (1) max_tokens=3000 is passed in every API call — the model cannot produce more than 3000 output tokens per response. (2) MAX_TOOL_ITERATIONS=6 caps the agent loop — if the model keeps requesting tools after 6 iterations, the loop exits with the last available content or a Malay-language fallback message. Input is not chunked or truncated; inputs exceeding the model's context window would fail with an API error (not yet encountered in testing).

2.4 Dependency Diagram — API Calls Between Components


Trader's Phone
Telegram
HTTPS polling (python-telegram-bot)
send_message()
getUpdates()
Telegram Bot API
Photo
Voice
Text
bot/handlers.py
services/ocr.py
EasyOCR
services/voice.py
Whisper
Direct Pass
extracted text
transcript
normalized input_text
agent/core.py - run_agent()
[system] + [history] + [user_msg]
POST /chat/completions + tools schema
ILMU API - api.ilmu.ai/v1
Z.ai / ilmu-glm-5.1
{context, tool_calls}
agent/tools.py - execute_tool()
db/queries.py
Supabase PostgreSQL / SQLite
Telegram Bot API
Open-Meteo API
loop
final text response

2.5 Sequence Diagram — Morning Brief → Approval → Execution

The flagship user journey, showing all system interactions from scheduler trigger to WhatsApp link delivery:



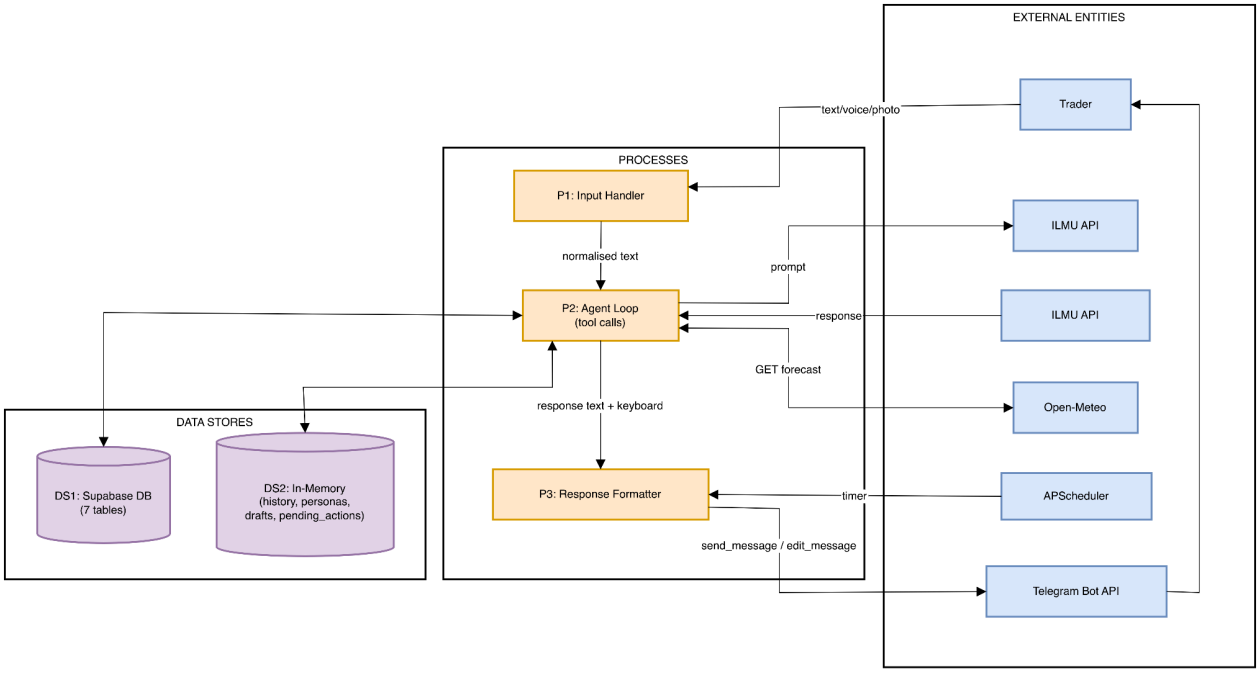
2.6 Technology Stack

Layer	Technology	Justification
Client / Frontend	Telegram Bot API (python-telegram-bot v20)	93% Malaysian smartphone Telegram penetration. No app install. Cross-platform. Supports inline keyboards, voice, photo.
Backend Runtime	Python 3.11 (asyncio)	Async I/O required for non-blocking DB + API calls. python-telegram-bot v20 is async-native.
AI / LLM	ilmu-glm-5.1 via ILMU API (Z.ai / YTL AI Labs)	Malaysian language fluency, tool-calling, multi-turn reasoning, cost-efficient vs GPT-4.
OCR	EasyOCR 1.7 (local)	Supports English + Malay, no API cost, runs in ThreadPoolExecutor for async safety.
Voice Transcription	faster-whisper (local, CTranslate2)	Faster than openai-whisper, CPU-efficient, no API cost, supports Malay.
Database (prod)	Supabase PostgreSQL + asyncpg driver	Managed PostgreSQL, free tier, row-level security, async driver for non-blocking queries.
Database (dev)	SQLite + aiosqlite	Zero-config local development. Same SQLAlchemy ORM layer — no code changes needed.
ORM	SQLAlchemy 2.0 (async)	Async session management, type-safe queries, supports both Postgres and SQLite.
Scheduler	APScheduler 3.10 (AsyncIOScheduler)	Integrates with asyncio event loop. CronTrigger for time-based proactive jobs.
Weather API	Open-Meteo (free, no API key)	Malaysian city support, 7-day forecast, zero cost, JSON REST.
HTTP Client	httpx (async)	Async HTTP for weather API calls. Replaces requests in async context.
Config	python-dotenv + config.py	Single config module with lazy loading — API keys read at call time, not import time.

3. Key Data Flows

3.1 Data Flow Diagram — System Level

External entities → Processes → Data Stores. Arrows show data movement direction.



DFD compliance note: No black holes (every process has output). No miracles (every process has input). External entities are: Trader, ILMU API, Open-Meteo API, APScheduler (time source).

3.2 Normalised Database Schema (3NF)

7-table schema. All tables in Third Normal Form — no transitive dependencies. Foreign keys enforce referential integrity.

Table	Primary Key	Columns (bold = FK)
inventory	id (int)	commodity (str), quantity_kg (float), entry_date (date), shelf_life_days (int), cost_per_kg (float), supplier_id → supplier.id
supplier	id (int)	name (str, unique), phone (str), language (str), reliability (float)
supplier_price	id (int)	supplier_id → supplier.id, commodity (str), price_per_kg (float), quantity_kg (float), quoted_date (date)
trade	id (int)	trade_type (buy/sell), commodity (str), quantity_kg (float), price_per_kg (float), counterparty (str), trade_date (date)

receivable	id (int)	buyer_name (str), commodity (str), amount_rm (float), due_date (date), paid (bool), paid_date (date)
fama_benchmark	id (int)	commodity (str), price_per_kg (float), week_date (date) — unique constraint on (commodity, week_date)
user_profile	id (int)	user_id (int, unique — Telegram ID), name (str), language (str), commodities (JSON str), city (str), updated_at (datetime)

3NF justification:

1. Every non-key attribute depends on the whole key (no partial dependencies — all tables have single-column PKs).
2. No transitive dependencies — supplier contact info (phone, language) is in supplier, not duplicated in supplier_price or inventory.
3. Junction pattern: supplier_price resolves the M:N between supplier and commodity×date.
4. Trade table is append-only (insert-only audit log) — no updates, ensuring data integrity for velocity and revenue calculations.

4. Functional Requirements & Scope

4.1 Minimum Viable Product (MVP)

Five core features demonstrated at UMHackathon 2026:

#	Feature	Description
1	Morning Brief + Approval → Execution	Scheduler fires at 3:30 AM. Agent runs 4-tool chain. Brief sent with ✅/⏭ keyboard. On approve: inventory updated + WhatsApp links generated per supplier in their language.
2	Photo OCR → Auto Inventory	Trader photographs delivery note. EasyOCR extracts text. ilmu-glm-5.1 parses commodity/qty/price/supplier. update_inventory called automatically. Zero manual entry.
3	Proactive Autonomous Alerts	6 APScheduler jobs: spoilage check (8AM/2PM), velocity alert (4-hourly), credit reminder (9AM), festival prep (10AM), weekly digest (Monday 7AM). All silent when no action needed.
4	Conversational Queries (10 tools)	Natural language interaction in Malay/English/Mandarin. Agent loop calls tools based on query: inventory, prices vs FAMA, credit, velocity, weather. Never answers from training data alone.
5	Stocky's Instinct	14-day cross-signal analysis (velocity + prices + credit + inventory timing). Surfaces ONE non-obvious pattern the trader likely missed. Appended to morning brief and weekly digest.

4.2 Non-Functional Requirements

Quality	Requirement	Implementation
Scalability	System must support multiple concurrent traders without shared state corruption	All state keyed by user_id (in-memory dicts + DB). AsyncIOScheduler iterates ACTIVE_USERS set. Async SQLAlchemy with connection pooling (pool_size=5). Stateless per-request design.
Reliability	Proactive jobs must not silently fail. Individual tool errors must not crash the agent.	Each tool call wrapped in try/except returning {error: str}. call_llm() has 2-attempt retry. Scheduler jobs log errors and stay silent rather than sending broken messages.
Maintainability	Codebase must be modular — each subsystem independently testable and replaceable.	5 modules (agent/, bot/, db/, services/, scheduler/), each with __init__.py and owner attribution. 27 test files with in-memory SQLite and mock LLM fixtures. conftest.py shared fixtures.
Token Latency	LLM API calls must return within acceptable time; UI must not appear frozen.	Async API calls via asyncio. Loading indicators shown to user ('⏳ Menjana brief...', '📸 Membaca gambar...'). max_tokens=3000 prevents runaway generation. 5s timeout implicit in httpx.
Cost Efficiency	Average inference cost per active user per day must be minimised.	Silence rules: scheduler jobs skip LLM call if data conditions not met. Local OCR+Whisper = zero vision/speech API cost. save_history=False for

		proactive calls prevents context growth. Prompt length capped by history trim (10 turns).
Security	API keys must not be exposed. Bot token must not conflict.	.env file loaded via python-dotenv with override=True. BOT_TOKEN validated at startup. deleteWebhook called on polling start to clear any webhook conflicts. Keys never logged.
Accessibility	System must be usable by traders with low digital literacy.	Telegram is the only interface — no new app. Voice note support for hands-free input. 4-question onboarding in plain language. Responses capped at 5 lines for mobile readability.

4.3 Out of Scope / Future Enhancements

- WhatsApp Business API direct sending — requires Meta approval process and monthly fees; replaced with wa.me deep links for hackathon
- Real-time FAMA price scraping — FAMA publishes PDFs only; a production version would parse weekly PDFs automatically
- Multi-tenant wholesaler accounts — current architecture is single-trader; production would scope scheduler jobs per user
- React dashboard web application — designed but not in core hackathon submission
- Supplier self-service portal — suppliers onboarded indirectly via trader interactions
- Predictive demand forecasting — Instinct currently detects patterns; future version would generate forward-looking predictions using time-series models
- Integration with Malaysian digital payment platforms (DuitNow, Touch 'n Go) for credit settlement tracking
- Multi-language within a session — language is fixed at onboarding; mid-session switching is future work

5. Monitoring, Evaluation, Assumptions & Dependencies

5.1 Grayscale Rollout & A/B Testing Strategy

For production deployment after the hackathon, the following phased rollout is planned:

- **Phase 1 (5% rollout):** Deploy to 5 traders at a single Pasar Borong location. Monitor: morning brief relevance (manual review), spoilage alert false positive rate, OCR extraction accuracy on real delivery notes.
- **Phase 2 (25% rollout):** Expand to 25 traders across 3 markets. A/B test: Group A receives full morning brief with all 4 tool chains; Group B receives simplified brief (inventory + weather only). Measure: order approval rate, time-to-approval, spoilage reduction.
- **Phase 3 (100% rollout):** Full deployment after 2-week Phase 2 monitoring with no P1 issues. Festival prep alert activated only after real-world validation of 1 festival cycle.
- **A/B test — Instinct feature:** Group A: morning brief with Instinct appended. Group B: morning brief without. Measure: 7-day pattern detection accuracy validated against trader retrospective feedback.

5.2 Emergency Rollback & Golden Release

Each deployment is tagged as a Git release. The 'Golden Release' is the last version with zero P1 incidents in a 48-hour production window.

- **Trigger:** P1 Critical incident detected (see Priority Matrix below). Automated pipeline redeploys Golden Release Docker image. APScheduler jobs resume from next scheduled trigger.
- **DEMO_MODE fallback:** If ILMU API is unreachable, setting DEMO_MODE=True in .env causes call_llm() to return mock responses immediately — the bot remains functional and responsive without any code changes.
- **Database rollback:** Supabase point-in-time recovery. SQLAlchemy schema migrations are additive-only (no destructive ALTER TABLE) to allow safe rollback.

5.3 Priority Matrix

Level	Trigger Condition	Threshold	Action
P1 Critical	ILMU API unreachable	3 consecutive failed calls	Switch DEMO_MODE=True. Alert dev team. Emergency rollback if API down >30min.
P1 Critical	Database connection lost	SQLAlchemy OperationalError on 3 queries	Bot returns maintenance message to users. Auto-reconnect on next query. Rollback if data corrupted.
P2 High	Morning brief not sent by 4:00 AM	Job completed flag not set by 4:00 AM KL time	Log error. Manual /trigger_brief morning fallback. Investigate scheduler.
P2 High	Telegram 409 Conflict	Duplicate polling instance detected	Kill all python.exe processes. Restart single instance. Add process lock file.
P3 Medium	OCR extraction failure rate >20%	3+ consecutive photo failures	Log failures with image hash. Review EasyOCR language config. Notify user to type manually.
P4 Low	Weather API timeout	httpx timeout on Open-Meteo call	Proceed with brief without weather context. Log. No user-facing error.

5.4 Monitoring Plan

For the hackathon phase, monitoring is via Python logging (stdout). Production monitoring plan:

- **Application logs:** Structured logging with %(asctime)s | %(levelname)s | %(name)s. All tool calls logged at INFO (name, args truncated to 120 chars). All errors at ERROR with exc_info=True.
- **Scheduler health:** APScheduler fires a heartbeat log on every job execution. Missing heartbeat detectable via log gap analysis.
- **API health:** call_llm() logs HTTP status code on each call. 2-attempt retry is logged. Consecutive failures trigger P1.
- **Cost monitoring (production):** Token usage logged per call. Daily rollup against budget threshold. Alert if daily tokens exceed 150% of baseline.

5.5 Assumptions

- Traders have a Telegram account and a smartphone with camera and microphone capability.
- Internet connectivity is available at the trader's location (Pasar Borong has WiFi/4G coverage).
- The ILMU API (api.ilmu.ai/v1) maintains OpenAI-compatible tool-calling format — no changes to the function_call schema.
- Supabase free tier connection limits (max 60 connections) are sufficient for the hackathon phase.
- Open-Meteo provides accurate forecasts for Malaysian cities (Kuala Lumpur, Klang, Shah Alam, Seremban, Ipoh, Johor Bahru).
- FAMA weekly benchmark prices seeded in the database represent realistic market price guidance for the demo period.
- The trader is the sole user of their Telegram account — no shared device scenarios in scope.

5.6 External Dependencies

Tool / Service	Purpose	Risk & Mitigation
Z.ai ILMU API (ilmu-glm-5.1)	Core LLM reasoning engine — prompt inference, multi-step tool-calling, response generation in Malay/English/Mandarin	HIGH: API down → core product fails. Mitigation: 2-attempt retry, DEMO_MODE=True fallback with mock responses, pre-recorded demo screenshots.
Telegram Bot API	Delivery channel — send/receive messages, inline keyboards, file downloads (voice/photo)	MEDIUM: Telegram outages are rare but global. Mitigation: drop_pending_updates=True to clear backlog on restart. Polling auto-retries.
Supabase PostgreSQL	Primary persistent data store — all inventory, trades, credit, prices, user profiles	MEDIUM: Free tier has 500MB storage and 60 connection limits. Mitigation: SQLite fallback for local dev. Connection pooling to stay within limits.
Open-Meteo API	Free weather forecast API — 7-day forecast for Malaysian cities, no API key required	LOW: Free tier rate limits. Mitigation: weather failure caught in execute_tool(), agent proceeds without weather context.
EasyOCR (local)	Image text extraction for delivery note photos — English + Malay, CPU-based	LOW: First run downloads ~100MB model weights. Mitigation: pre-warm before demo. Fallback: trader types manually.
faster-whisper (local)	Voice note transcription — Malay and English, CTranslate2 optimised	LOW: Requires ffmpeg system dependency. Mitigation: install documented in README. Fallback: trader types.
WhatsApp wa.me links	Pre-written supplier order message delivery — deep links open WhatsApp with pre-filled text	LOW: wa.me links require WhatsApp installed on trader's phone. Not a sending API — no approval needed.

6. Project Management & Team Contributions

6.1 Project Timeline (10-Day Sprint)

Days	Sprint Goal	Deliverables
Day 1–2	Architecture & Planning	System architecture finalised. DB schema designed (7 tables, 3NF). Tech stack confirmed. Module ownership assigned. config.py and db/models.py built. Requirements.txt locked.
Day 3–4	Core Agent Loop	agent/core.py: run_agent() with tool-calling loop. services/glm.py: ILMU API wrapper with retry. agent/tools.py: 10 tool schemas + dispatcher. agent/prompts.py: BASE_SYSTEM_PROMPT with mandatory tool chains. db/queries.py: full CRUD layer.
Day 5–6	Bot Handlers & Proactive Scheduler	bot/handlers.py: text, voice, forwarded message routing. agent/persona.py: 4-question onboarding + persona injection. scheduler/jobs.py: 6 APScheduler cron jobs. Morning brief + instinct layer. bot/keyboards.py: approval keyboards.
Day 7–8	Advanced Features	services/ocr.py: EasyOCR pipeline + photo handler. services/voice.py: faster-whisper transcription. services/festivals.py: 2025-2026 Malaysian festival calendar. festival_prep_job + /trigger_festival demo command. Approval → execution loop with WhatsApp links.
Day 9	Testing & Debugging	27 test files written. conftest.py with shared fixtures. All call_glm → call_llm bugs resolved. Tool schema tests fixed (get_instinct_analysis → mark_credit_paid). Weather result wrapping fix. 409 Conflict resolution documented.
Day 10	Documentation & Demo Prep	PRD and SAD written. Demo script prepared (6 scenes, 10 minutes). /trigger_festival and /trigger_brief commands verified. EasyOCR model pre-warmed. Supabase demo data seeded. run.bat created.

6.2 Team Member Roles

Member	Role	Key Deliverables
Person 1	AI/Agent Lead	agent/core.py, agent/tools.py, agent/prompts.py, agent/instinct.py, services/glm.py — the full LLM integration stack
Person 2	Database Lead	db/models.py, db/queries.py, db/seed.py — all 7 tables, 14 query functions, demo data seeding
Person 3	Bot & UX Lead	bot/handlers.py, bot/keyboards.py, bot/formatters.py — Telegram routing, OCR pipeline, approval→execution loop, WhatsApp link generation
Person 4	Scheduler & Services Lead	scheduler/jobs.py, services/weather.py, services/festivals.py, services/voice.py — all 6 proactive jobs, weather/festival integrations

Person 5	QA & Testing Lead	tests/ (all 27 files), conftest.py, test fixtures — unit tests for every module, integration test scenarios, bug detection and resolution

— End of Document —

Stocky AI | UMHackathon 2026 | Powered by Z.ai / YTL AI Labs